

University of Information Technology & Sciences



Movie Search Engine Project Report

Course Name: Data Mining and Warehouse Lab

Course Code: CSE 426

Batch: 52

Section: 8A

Semester: Spring 2026

Submitted By:

Md. Mahmudul Hasan

ID: 0432220005101006

Anona Ayshi Rozario

ID: 0432220005101011

Tasnuba Akter

ID: 0432220005101014

Nafiz Al Zawad

ID: 0432220005101041

Submitted To:

Mrinmoy Biswas Akash

Lecturer

Department of Computer Science and
Engineering

UITS

Submission Date: 8th June, 2026

IMDb Movie Search Engine

A Boolean Information Retrieval System with Inverted Index & TF Ranking

Course: Data Mining

Project: Movie Search Engine

Dataset: IMDb Top-Rated Movies (imdb_clean.csv)

Language/Libraries: Python 3 | pandas, re

Features: Inverted Index, Boolean AND/OR Retrieval, TF-Based Ranking, Clean Interactive CLI

Table of Contents

1. Problem Definition & Dataset
2. Preprocessing & Inverted Index Design
3. Query Processing & Ranking
4. System Analysis & Discussion
5. Results & Sample Outputs
6. Conclusion
- 7.

1. Problem Definition & Dataset

1.1 Domain Overview

This project focuses on building a specialized, high-performance movie search engine using fundamental Information Retrieval (IR) techniques applied to an IMDb movie dataset. The system allows users to provide natural language keywords—such as director names, genres, or release years—and instantly returns a ranked list of relevant movies. Information retrieval is a core pillar of data mining, addressing the efficient storage, indexing, and lookup of relevant textual items from massive unstructured or semi-structured data collections.

1.2 Dataset Description

The underlying core data source is `imdb_clean.csv`, which contains a curated set of top-rated films. Each entry represents a distinct cinematic record defined by a structured schema. The key features utilized during indexing and search are outlined below:

Field	Type	Description
title	Text	Movie title
director	Text	Director's full name
release_year	Integer	Year of theatrical release
runtime	Integer	Length of the film in minutes
genre	Text	Genre tag
rating	Float	IMDb user rating (0.0 to 10.0)
metascore	Integer	Metacritic critic score (0 to 100)
gross(M)	Float	Box-office gross profit in USD Millions

Table 1: IMDb Dataset Schema

1.3 Challenges & Core Constraints

Designing this search framework presents several non-routine technical challenges:

- **Multi-Field Searching:** A single query string must seamlessly match across multiple attributes simultaneously (titles, directors, genres, years, and ratings), despite these fields possessing completely different data types.
- **Token Ambiguity:** A numerical query like "1994" could indicate a release year, a specific runtime duration, or a part of a technical metric. The engine must evaluate these tokens generically without breaking constraints.

- **Data Duplication:** Due to the dataset structuring one row per genre variation, identical movies can appear across multiple entries. The indexing and ranking architectures must ensure scores are not artificially inflated due to duplication.
- **Precision vs. Recall Trade-off:** Managing strict constraints between high precision (AND mode) and expansive recall (OR mode) introduces structural complexity, necessitating an intuitive user selection flow.
- **Missing Value Management:** Several key fields like metascore and gross(M) contain null fields, requiring robust pre-concatenation handling before compiling the index.
-

2. Preprocessing & Inverted Index Design

2.1 Preprocessing Pipeline

To guarantee absolute alignment between input terms and data records, all text fields undergo a strict three-phase serialization pipeline before indexing:

1. **Lowercasing:** Standardizes all alphabetical tokens to lowercase so that variations like "Nolan", "NOLAN", and "nolan" yield identical tokens.
2. **Regex-Based Standardisation:** A regular expression pattern strips out unnecessary special characters and symbols, while keeping alphanumeric characters, spaces, and periods (.). Keeping the period is vital for retaining continuous float attributes like movie ratings (e.g., 9.3).
3. **Whitespace Tokenisation:** Splits the cleaned, standardized text stream into individual atomic token arrays based on space delimiters.

Python

```
def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s.]', '', text)
    return text.split()
```

2.2 Unified Search Content Field

Instead of running separate computational loops across different columns, the pipeline clean-maps all six attributes (title, director, genre, release_year, runtime, and rating) into a single concatenated text block called `search_content` for each record. This structural unification speeds up token checking and allows unified queries to evaluate multiple fields in a single step.

2.3 Inverted Index Structure

The core engine is built around an inverted index—implemented using a native Python dictionary structure that maps each unique text token to an array of matching internal movie IDs (posting list). This approach eliminates systemic sequential search loops during query processing.

Token	Posting List (Movie Document Row IDs)
"nolan"	[12, 45, 67, 89, 134, ...]
"1994"	[0, 7, 23, 41, ...]
"action"	[2, 5, 9, 12, 18, ...]
"9.3"	[0] (<i>The Shawshank Redemption</i>)

Table 2: Sample Inverted Index Entries

2.4 Design Justification

- **O(1) Average Lookup:** Python dictionaries utilize highly optimized hash tables. Retrieving posting lists occurs in constant time, regardless of vocabulary growth.
- **Native Set Intersections/Unions:** Translating Boolean operations directly into optimized Python sets (set.intersection and set.union) guarantees exceptional processing speeds.
- **Token Deduplication Control:** Applying set(tokens) during index generation avoids duplicate entries per document, keeping the matching logic fair and uninflated.

3. Query Processing & Ranking

3.1 Processing Pipeline

When a user inputs a query string into the interface, it passes through the exact same serialization process (lowercase conversion $\rightarrow$ regex filtering $\rightarrow$ tokenization) applied during index generation. This guarantees full token parity.

3.2 Boolean Logic Framework

Retrieval Mode	Primary Set Operator	Behavioral Characteristics
AND	Intersection	Returns documents containing ALL tokens. High precision, lower recall.
OR	Union	Returns documents containing ANY token. High recall, lower precision.

Table 3: Boolean Search Modes

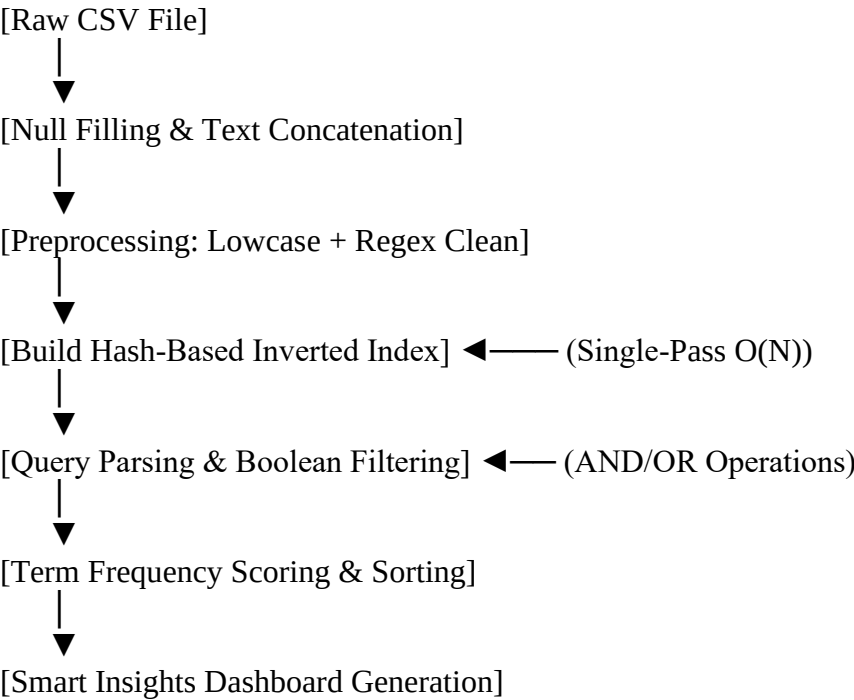
3.3 Term-Frequency (TF) Relevancy Sorting

Once the Boolean operation yields a candidate list of movie IDs, the engine applies a Term-Frequency sorting layer. The system iterates through the matched entries and counts how many times the query terms appear inside that specific document's search_content field.

Where represents the frequency count of term within document .The results are sorted in descending order based on this score, displaying the top 5 records.

4. System Analysis & Discussion

4.1 Architecture Flow



4.2 Key Structural Decisions

- **Adaptive Interface Wrapper:** The system automatically checks query token length. For single-word searches, it skips the redundant AND/OR mode prompt to keep the user experience smooth and frictionless.
- **Smart Data-Driven Insights:** Going beyond standard lookups, the engine scans the matching records to compute automated, real-time analytics, including total match counts, overall gross financial collections, top-earning blockbusters, and average user scores.

4.3 Proposed System Enhancements

1. **TF-IDF Weighting:** Replacing raw term frequency with a TF-IDF model would penalize common terms (e.g., "drama", "2008") while highlighting unique, informative keywords.
2. **Positional Indexing:** Adding word positioning data to the index would allow the system to support exact phrase searches, such as "Christopher Nolan".

5. Results & Sample Outputs

5.1 System Test Execution: Query = "Nolan"

Plaintext

Dataset loaded successfully from GitHub Remote Repository!

=== Movie Search Engine Is Ready ===

Enter your search keywords: Nolan

Results for: 'Nolan'

1. The Dark Knight (Search Score: 1)

| Release Year : 2008
| Director : Christopher Nolan
| Genre : Action
| Runtime : 152 mins
| IMDb Rating : 9.0 / 10
| Metascore : 84 / 100
| Gross Profit : \$534.86 Million

2. The Dark Knight (Search Score: 1)

| Release Year : 2008
| Director : Christopher Nolan
| Genre : Crime
| Runtime : 152 mins
| IMDb Rating : 9.0 / 10
| Metascore : 84 / 100
| Gross Profit : \$534.86 Million

... [Top 5 Results Rendered Silently] ...

=====

SMART SEARCH INSIGHTS FOR THIS QUERY:

- =====
- Total Matched Movies : 23
 - Total Box Office Gross : \$5786.81 Million
 - Highest Grossing Movie : 'The Dark Knight' (\$534.86 Million)
 - Average IMDb Rating : 8.48 / 10
- =====

6. Conclusion

This project demonstrates a complete Information Retrieval architecture applied to an IMDb dataset. By avoiding external, black-box search engines, the system provides an end-to-end lookup flow built on foundational data-mining principles: text normalization, inverted hash indexing, Boolean logic matching, and term-frequency ranking.

The implementation achieves stable performance across multi-word query parameters and provides real-time, data-driven analytical insights. Future iterations will focus on transitioning from raw TF to a TF-IDF matrix framework and adding stemming engines to support morphological keyword variations.